

A multiple case study on the inter-group interaction speed in large, embedded software companies employing agile

Antonio Martini^{1,2,*}, Lars Pareto² and Jan Bosch²

¹*Software Engineering, Chalmers University of Technology, Gothenburg, Sweden*

²*Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden*

ABSTRACT

The adoption of Agile Software Development in large companies is a recent phenomenon of great interest both for researchers and practitioners. Although intra-team interaction is well supported by established agile practices, the critical interaction between the agile team and other parts of the organization is still unexplored in literature. Such interactions slow down the development, hindering the achievement of business goals based on speed: short time to market, quick replication of products of a product-line, and reaction time for product evolution.

We have employed a two-year long multiple-case case-study, collecting data through interviews and a survey in three large companies developing embedded software. Through a combination of qualitative and quantitative analysis, we have found strong evidence that interaction challenges between the development team and other groups in the organization hinder speed and are widespread in the organizations.

This paper also identifies current practices in use at the studied companies and provides detailed guidelines for novel solutions in the investigated domain. Such practices are called boundary-spanning activities in information system research and coordination theory. We present a comparison between large embedded software companies employing agile and developing a line of products based on reused assets and agile companies developing pure software. We highlight specific contextual factors and areas where novel spanning activities are needed for mitigating the interaction challenges hindering speed. Copyright © 2015 John Wiley & Sons, Ltd.

Received 7 August 2014; Revised 23 September 2015; Accepted 16 October 2015

KEY WORDS: inter-group interaction; Agile Software Development; development speed; boundary-spanning activities; embedded software; software product line

1. INTRODUCTION

Large software industries strive to make their development processes fast and more responsive, minimizing the time between the identification of a customer need and the delivery of a solution. Agile Software Development (ASD) gives support for such goals and has proven successful in small software projects in the last decade [1]. Less evidence can be found on its success in large software companies, where research is still ongoing [2]. The adaptation of ASD to embedded software, driven by an overall physical product development process [3], seems to be even more challenging and unexplored.

ASD stresses the importance of responsiveness, and the continuous interaction between actors involved in the software development process. In large software companies, an agile team needs to interact with a number of groups: the customers (or their surrogates, i.e., product owners) or other stakeholders, such as hardware development teams or system architects; suppliers [4]; and other

*Correspondence to: Antonio Martini, Software Engineering, Chalmers University of Technology.

†E-mail: antonio.martini@chalmers.se

agile or non-agile teams [5]. Some studies highlight how interaction among several social groups is often a critical factor, hindering the speed benefits gained by the parallelization of software development [6–9]. Specifically, several interaction challenges have been found to hinder three business goals dependent on speed: first deployment speed, replication speed, and evolution speed [8].

However, there is a need for understanding which boundaries exist between the agile team and other groups with the surrounding organization and which interaction challenges are more relevant [1, 10], in particular for the embedded software domain. Existing theories suggest the need for boundary-spanning activities in order to establish a coordination strategy [11–13]. Such activities are meant to fill the interaction gap existing between two social groups by spanning the existing boundary.

We employed a two-year long case-study involving three firms to understand which critical interaction challenges hinder achieving business goals dependent on development speed and how widespread such challenges are in the companies. We have also inquired the current status of practice regarding spanning activities at the studied companies. This paper focuses especially on large embedded software development companies employing ASD. We hypothesize a significant difference between embedded and pure software cases reported in literature. In relation to context, we aim at answering the following research questions:

- RQ1 Which inter-group interaction challenges influence business goals dependent on speed in large-scale software development?
- RQ2 Which inter-group interaction challenges are more critical and widespread in the organizations?
- RQ3 Which spanning activities for mitigating the interaction challenges are currently employed or missing by practitioners?
- RQ4 What are the differences between embedded software companies and pure software companies in terms of interaction challenges and spanning activities?

This paper has three parts and the main contributions are the following:

- A list of interaction challenges hindering several business goals dependent on speed. The list is ranked based on the level of recognition (strong, controversial, and weak) and the spread through the companies. The challenges also show critical organizational boundaries and are discussed in relation to companies' contexts and participants.
- A set of spanning activities, objects, and coordinator roles that are used by the participants or are missing. Such activities can be used for the creation or adaptation of a coordination strategy that will improve speed by mitigating the interaction challenges and therefore increasing speed.
- A comparison of our selected cases (embedded software domain) and the pure software cases analyzed in literature focused on the same topic. We show a significant difference in the contextual factors and the areas where spanning activities are needed.

In the next section we describe our conceptual framework and present the theoretical components on which we built our research: speed, social interactions, spanning activities, and ASD. Then we describe our methodology, a two-year long multiple-case case-study including three firms, in which we have employed both qualitative and quantitative methods. We then present our results and their analysis, in which we show the previously mentioned contributions. The following section will discuss the generalization of the main results, how they inform the research questions, and how they enrich the current body of knowledge in software engineering. The paper ends with the conclusions and our targets for future research.

2. BACKGROUND

Figure 1 shows our overall conceptual framework: in the observed software companies we found *interaction challenges* between the agile teams and other social groups. Such challenges hinder three business goals dependent on three kinds of speed: *first deployment speed*, *replication speed*, and *evolution speed* [8] (shown by 'dashed' arrows in the picture). Such interaction challenges can be mitigated by the application of *spanning activities*. The results will be presented using this

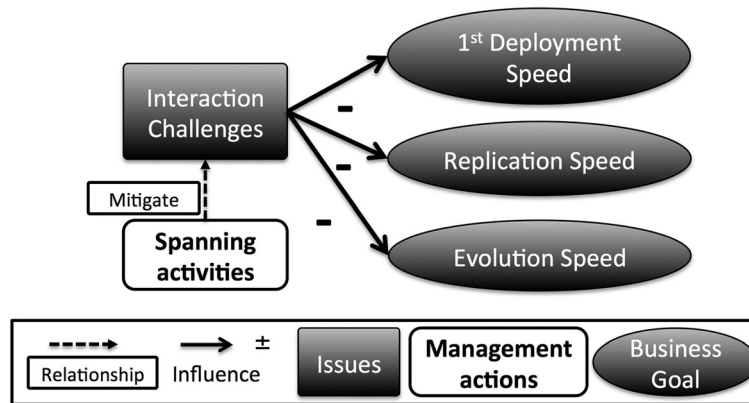


Figure 1. Overall conceptual framework for managing inter-group interaction speed.

framework: the discovered interaction challenges, which goal they affect and which spanning activities are present or are missing in the observed companies.

2.1. Speed

Many studies dealing with speed, e.g. [6–9, 14–16], identified interactions among several actors in the software development process as a major generic influencing factor. Despite such recognition, we have found no studies that tackle this problem in depth and in relation with speed.

According to previous research results [5, 8], large companies developing multiple products with a substantial amount of embedded software need to manage three end-to-end speeds (Figure 1): the speed with which customer needs lead to new product offers (*first deployment speed*), the speed with which new features are replicated in new products (*replication speed*), and the speed with which change request to an existing product are realized (*evolution speed*). Lean Software Development also stresses the importance of end-to-end speed for the release of a product over the local speed of a single team, in order to optimize the whole instead of local output [17, 18]. Such principle differs between Lean and ASD, where instead the team speed is emphasized, which implies a distinction in companies' development approaches in practice [1, 19]. However, several mixed 'Leagile' approaches have been applied according to the cases studied by Pikkarainen *et al.* [19], showing that the combination of principles is possible. For this reasons, we borrow the idea of waste elimination from Lean literature, where speed has been *defined* as 'absence of waste' [17, 18]. Therefore, we investigate the source of such speed waste, more specifically the speed waste because of interaction challenges (refer to the next section) between the agile team and other parts of the organization hindering the previously mentioned end-to-end business goals. We also aim at finding possible solutions for such challenges: the results might be used to complement existing agile practices, used by the teams to achieve local speed, with activities that would benefit the end-to-end speed according to the Lean principles.

2.2. Social interactions in Agile Software Development

One important element in ASD is interaction, which also influences both the delivered value and the time spent on coordinating all the actors in a large organization.

The *development team* is a *social group* (*Group dynamics research and theory*, 1953) of interdependent employees responsible for the development of a portion of software, which might be a feature or a component. Other social groups may be a test unit, a supplier, a management team, etc. Between two social groups stands an *organizational boundary* [20]. *Interaction challenges* may occur between the development team and any other group in the organization. These challenges might hinder the achievement of speed or other business goals, as reported in several studies [6–9, 14–16]. However, the lack of in-depth studies on such interaction challenges, especially related to business goals dependent on speed, motivated our research questions:

- RQ1 Which inter-group interaction challenges influence business goals dependent on speed in large scale software development?

- RQ2 Which inter-group interaction challenges are more critical and spread?

2.3. Interaction challenges and spanning activities

In order to mitigate the interaction challenges, it is important to understand how the practitioners currently avoid such issues and therefore what software development practices are still missing. Such practices involving several social groups within an organization have been recently referred as spanning activities in information system research [11]. Such concept has been used by Strode *et al.* [13], who proposed the use of the current results from coordination theory [12] and analyzed how co-located agile projects coordinate. Among the coordination mechanisms, they mentioned spanning activities as follows: ‘*Boundary spanning occurs when someone within the project must interact with other organizations, or other business units, outside of the project to achieve project goals. There are three aspects to boundary spanning: boundary spanning activities, the production of boundary spanning artifacts, and coordinator roles*’.

Such definition allows us to consider interaction challenges within the broader scope of project coordination: we take inspiration by Strode’s framework for analyzing and organizing our results and for comparing our cases with the only reference in literature concerned, although only partially, with spanning activities.

Strode’s framework proposes the definition and description of *spanning activities*, of *spanning objects* (artifacts) the *coordinators* (roles) responsible for carrying them out in the organization. Each activity needs to be carried out also at a specific frequency, i.e., *per-project*, *per-iteration*, and *ad-hoc*. Our results might be used in the creation of an overall coordination strategy, which is, however, beyond the scope of this paper.

According to the previous definition and the findings highlighted by Strode *et al.* [13], Levina and Vaast [11] and Malone and Crowston [12], it’s necessary to establish spanning activities for mitigating interaction challenges. This leads to our next research question:

- RQ3 Which spanning activities for mitigating interaction challenges are currently employed or missing by practitioners?

2.4. Agile in large-scale embedded software development

Embedded software is developed to control machines or devices that are not considered generic purpose computers. Such software is typically specialized for the particular hardware that it runs on and has to match specific time and memory constraints. Examples from such domains are cars, telephones, modems, robots, appliances, toys, security systems, pacemakers, televisions and set-top boxes, and digital watches.

ASD had a great impact on software development and has been the subject of researchers’ attention in recent years [1]. Companies developing software for specific domains, such as embedded software development, have shown a trend to adopt ASD [3, 21] However, such development process shows properties and difficulties that are not common to generic software development, and which have to be taken care of when employing ASD. For example, agile teams depend on strict hardware requirements and resources and the necessity of following the overall product development [3, 21, 22].

Consequently, we make the hypothesis that the embedded software domain would bring more and/or different interaction challenges, and therefore, it would require different boundary-spanning activities. The aim is to answer RQ4:

- RQ4 What are the differences between embedded software companies and pure software companies in terms of interaction challenges and spanning activities?

3. METHODOLOGY

In this section we describe our strategic and tactical research decisions: we conducted a multiple-case case-study as a tool for data collection and the combination of quantitative and qualitative data analysis, as recommended by Yin [23].

3.1. Conceptual framework

The components of our conceptual framework described in the previous section were continuously updated with inputs from empirical studies and literature reviews. We have combined inductive and deductive approaches for theory building, as described in Dubois *et al.* [24] and as an essential feature of the Grounded Theory approach [25, 26].

3.2. Companies' contexts

The case selection followed a replication logic strategy, a tactic recommended in case study design [23], which can consist of either choosing cases that are similar and therefore likely to provide similar results (*literal replications*) or choosing cases that are dissimilar and therefore likely to provide contrasting results for predictable reasons (*theoretical replications*). The cases that we have selected represent literal replications, and therefore, we expected to produce results that were similar. On this regard, all the companies in the sample were developing embedded software, and we aimed at strengthening the results by inquiring several sources (*source triangulation*) as recommended in Runeson and Höst [27]. Our cases can, however, be considered theoretical replications when highlighting the differences compared with the ones reported in literature: to fulfill this purpose, we will discuss our cases with the ones included in Strode *et al.* [13].

Our cases were three large product-developing companies (A, B, and C), all with extensive in-house embedded software development. All were situated in the same geographical area (Sweden), and active on several international markets. Table I shows the context details for each of them. As for the Organization fields, it is important to notice that although the actual organizations were large, including hundreds of employees, we needed to focus our research to units of analysis [23] that would make our investigation feasible.

Table I. Studied cases and different context parameters.

	Case A	Case B	Case C
Product description	Several components of automotive embedded software.	Communication system, signal elaborating system, GUI system and customized OS platform.	Components and features development for a telecommunication platform.
Product lifecycle	6–7 years	>15 years	~15 years
Business Model	International markets	International markets and individual customers	International markets
Processes	• Scrum based implementation – several weeks iterations • Input: requirements from different product owners/ different projects • Output: deliver a <i>platform</i> to testers to be validated and verified	• Scrum based implementation – several weeks iterations • Input: specification System Engineers • Output: delivers a <i>subsystem</i> to separate unit of testers to be validated and verified	• Scrum implementation – few weeks iterations • Input: white box specification from System Engineers • Output: deliver a component to be integrated by other units developing other products
Organization (unit of analysis only)	• 10–20 developers (including architects) • Cross-functional teams • Testers in a separated team • Requirement Engineers in separated business unit	• 2–25 developers (depending on the project, including architects) • Organized in functional teams • Testers in separated team • System Engineers in separate team	• 20 developers (including architects) • Cross-functional teams • System Engineers in separated team
Outsourcing	Some components outsourced	No outsourcing	No outsourcing

Company A was involved in the automotive industry. Some components were developed by in-house teams following Scrum, while others were outsourced to suppliers. The surrounding organization was following a stage-gate release model. Business was driven by the development of products for mass customization.

Company B was a manufacturer of product lines of utility vehicles. In this environment, the teams were partially implementing ASD (Scrum). Some competences were separated; e.g., System Engineers sat separately and deliver specification written on documents. Special customers requesting special features drove the business, and speed was important for the business goals of this company.

Company C was a manufacturer of telecommunication systems product lines. Their customers receive a platform and pay to unlock new features. The organization was split into cross-functional teams, most of which with feature development roles. Some of the teams had special supporting roles (technology, knowledge, architecture, etc.). Most of the teams used their preferred variant of ASD (often Scrum). Features were developed on top of a reference architecture, and the main process consisted of a pre-study followed by few (~3) sprint iterations before the feature was deployed.

3.3. Data collection

In our data collection we used three steps, in which we alternated several methodologies. First we employed a set of in-depth interviews for exploring the challenges in the achievement of goals based on speed. In the second step we validated and prioritized the interaction challenges through the collection of quantitative data using a survey. We also used the survey for collecting qualitative data and exploring which spanning activities were employed or missing in the companies. Finally, we collected validation responses through focus groups.

3.3.1. Problem investigation: what challenges are decreasing speed? During the first step we conducted seven in-depth, semi-structured 2-h-long interviews with informants from each company. Semi-structured interviews are usually suited for exploratory studies, in which the researchers need to gain new insights on a novel phenomenon and strive to understand the factors involved. [23, 28].

Informants were chosen on the basis of their role and expertise, and the aim was to understand the issues by inquiring key employees rather than collecting a large amount of more superficial data. This allowed us to ask follow-up questions and therefore to follow multiple leads. All informants were senior engineers or managers, and had experience with the implementation of agile processes. For case A we interviewed an architect and a line manager from the same project. For case B, we interviewed four developers from four projects, some of whom had also architect or product line management roles. In case C, we interviewed one informant with the role of both senior developer and architect.

The interview protocol had fixed questions about the informant's role and context, and open-ended questions to bring out experiences related to challenges when reaching several kinds of speed.

Interviews followed the natural flow of discussion, and we included questions to cover predefined themes. Two main themes in the interview guide were represented by 'agile practices' and 'reuse practices'. Both of them were meant to capture key practices clearly aimed at different business goals based on speed.

3.3.2. Validation, prioritization of the challenges, and missing practices. The challenges revealed by the previous collection method, which relied only on qualitative information, needed to be strengthened by a more quantitative validation. Also, we wanted to understand which of the challenges were more severe in the companies and therefore needed to be prioritized. Finally, we wanted to understand what practices (spanning activities) were used or missing in different contexts.

For this purpose we created an online questionnaire that was designed using a combination of closed- and open-ended questions [29]. The first version of the survey was tested by two PhD students, to have feedback from an academic point of view, and by at least one contact in each company, to make the questionnaire fit the jargon and being understandable for all the respondents. During the test we also monitored time and caught misunderstandings because of formulation of the questions.

The motivation for using an online questionnaire was to maximize coverage and participation without having to set up a time-consuming interview system. A survey is a suitable method for data collection when asking structured questions [28, 29]. The participants could access the survey whenever they wished. The answers were bound to a web-link and a mail address, so the respondents could also interrupt the survey and continue it in another moment. The questionnaire was made accessible online at surveymonkey.com between April and June of 2012.

The questionnaire was structured into two parts:

- Two questions about the role of the respondents and their organizational unit of work. Different names, dependent on the context, were reported for the same role. We decided to abstract the wide set of roles into ‘manager’, ‘system engineer’, ‘designer’, and ‘tester’. We have chosen this categorization because of the frequent mention of such roles during the interviews, and we could easily map these roles to clear references in software development literature.
- A set of 23 interaction challenges. For each one, the respondents were asked to give an answer among four alternatives: ‘No, I don’t recognize the challenge’, ‘Yes, I recognize the challenge in other units’, ‘Yes, I recognize the challenge in my own unit’, and ‘Yes I recognize the challenge in my unit and other units’. When appropriate, we added a fifth answer, in which the respondents could express their inability to answer because of a lack of experience of the described situation. Such alternatives allowed us not only to validate the answers but also to obtain the perceived spread of the challenges in the organization. For each challenge we added an open-ended question using a text area. We asked which practices were applied or missing by the practitioners in order to handle the challenge.

The 23 challenges are the results from the previous exploratory interview study conducted in the same companies, in which such challenges were categorized as related to interaction. We operationalized the factors in a closed-question-survey. In some cases we decided to split the factor into two questions. For example, the inhibitor ‘different tool and programming languages’ was split into two questions, Q17 and Q18. For most of them, however, there is a one-to-one correspondence between the factors and the questions in the survey.

A 22-page survey guide with detailed explanation for most of the factors, complemented with concrete examples, was made accessible for the practitioners through the web.

The challenges, which represent also the results collected in the first step, are outlined in Table II.

3.3.3. Sample for the survey and response rate. We obtained 36 complete answers. The web tool helped us to select only the answers that came from respondents who finished the whole survey; i.e., incomplete answers have been discarded.

We had an overall response rate of 68% for the quantitative answers, i.e., 45 out of 66 total email invitations sent out, of which 36 were completed answers. This entails that 80% of the participants who started the survey finished it, which was, in the end, 54% of the total invited participants.

Figure 2 shows the distributions of answers across roles. We can see how all roles were covered: company B and C respondents cover all roles, whereas company A respondents cover all roles except testers. We notice a major presence of designers, especially for company C.

3.3.4. Validation of spanning activities: collective interviews. In order to validate our results, we organized three collective interviews, one at each company. Most of the practices were recognized, and we have excluded the ones rejected by the employees.

3.4. Data analysis: combination of qualitative and quantitative analysis

We analyzed the data using a combination of qualitative and quantitative approaches (*methodological triangulation*), as recommended in Runeson and Höst [27]. This choice is also regarded as ‘strong analytic strategy’ in Yin [23].

3.4.1. Grounded theory. For the qualitative data, the interviews from the first step and the open-ended answers in the survey, we used an approach based on Grounded Theory [25, 26]. Instead of starting with a hypothesis and then testing it, it is based on a bottom-up process in which the theories (and the hypotheses) emerge from the data and therefore are grounded in it. The aim is to avoid the

Table II. Challenges found in the first step and in-depth studied in the second step.

Q01	The processes/ways of working that you have to follow are not suited for the kind of product that you are developing
Q02	The processes/ways of working that you have to follow are not suited for the project management
Q03	Project-related defects (defects in the same reused component fixed in some project but not in others)
Q04	Developers and system engineers are not co-located, which causes communication problems in requirement agreement
Q05	There is an upcoming product. Erroneous assumptions have been made on what part of the existing software can be reused and/or adjusted, causing inaccurate budget or resources allocation
Q06	Evaluation of costs and feasibility of new features do not take into account implementations and constraints, causing inaccuracy in budget and resources (for example time or workload) allocation
Q07	There is no time to improve parts of software shared among projects.
Q08	A satellite unit is 'invisible': for example, it is difficult to consult them; there is no clear information on their work or cannot be guided properly (the kind of problem may depend on your role)
Q09	A development unit has to build a component to be integrated in other units' (projects') system, but the communication with them is not sufficient.
Q10	A development unit was forced to integrate a common component. This caused communication problems, and now the unit is not willing to integrate new common components.
Q11	Reuse is not supported by the Product Line Management, it is an individual initiative.
Q12	Different attitudes and values of distributed (not co-located) teams (or units, projects) caused communication problems.
Q13	Team's (or unit, project) lack of will to integrate a common component.
Q14	Lack of understandable documentation or of proper communication causing long warming up periods for consultancy or for new employees to understand the system
Q15	Documentation is abundant but it does not help to understand the code
Q16	Leaders' mindset is not open to listen and they are not able to recognize strengths and weaknesses. This hinders the development of improvements.
Q17	Different favorite programming languages in the same team create communication .
Q18	Different favorite tools in the same team create communication problems.
Q19	Artifacts received from the system engineers are not clear enough because they were created with inappropriate tools
Q20	Developers are too constrained by system engineers on design (for example, developers receive white box/very detailed specification)
Q21	An internal interface had to be exposed to other units to allow their development. The interface documentation provided did not help to understand the code.
Q22	Disagreement between different development units (or projects) about what set of functionalities a common component should provide. This creates communication problems
Q23	Loss of knowledge about a reused framework's variation points, for example a framework created some years.

researchers to fit the data to preconceived hypotheses. The method used is also focused on validating theories by the systematic comparison across the data, the codes, and the concepts.

We coded the text using a tool for qualitative analysis (Nvivo), which allowed us to maintain traceability between the data and the produced codes. The codes were categorized and sorted in order to link, compare and combine them into an organic representation of the investigated data, which led us to identify novel theories or to confirm existing knowledge.

3.4.1.1. Coding. We first analyzed the audio from the interviews and the text from the open-ended answers in the questionnaire, slicing it into small pieces and linking the relevant ones to quotations and codes. This task was carried out by the first researcher and checked by the second one and was part of the *open coding*, in which we followed Strauss and Corbin's method [26]. In this phase, the first researcher summarized and interpreted the informants' words. Some factors were the result of *in vivo* coding, where we quoted the interviewee; this was done in cases where the concept was well explained: '*Improvements depend on leaders mindset, open to listen and recognize strengths and weaknesses*'. For other factors we summarized from the source, '*Long warming up periods for consultancy*' summarized '*[...] is very hard for a consultant coming in [...] we see that we have a half-year initial period just for [tool name] tool; it is another half year or year for the product to come into that to know it on a pretty good level [...]*'. Yet other factors are referring to many statements, which are showing the specified clear issue, only if considered together: '*Business side afraid of upfront investment for a software product line*'.

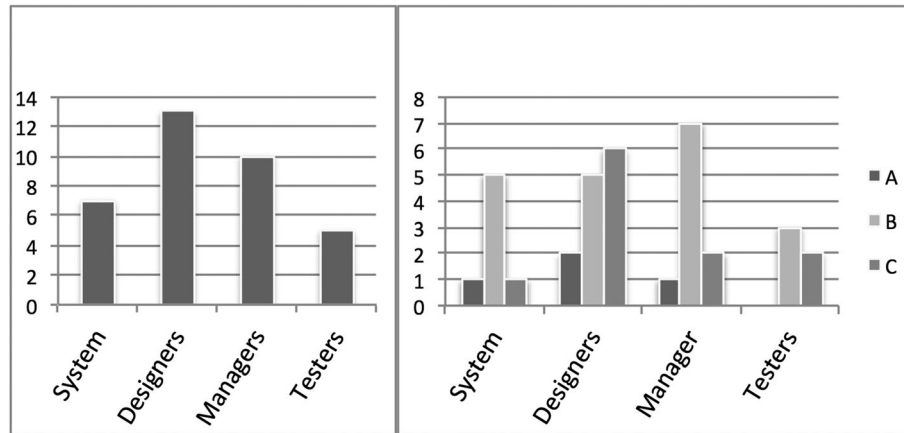


Figure 2. Sample of the respondents, divided by roles and companies.

3.4.1.2. Categorization. We then categorized the codes according to our research questions. In the first step, dedicated to problem identification, the categorization revealed the need for several hybrid categories for special influences, such as influencing several kinds of speed with different polarities: for instance, ‘Lack of detailed documentation’ has a negative influence on *first deployment speed* but a positive influence on the *Replication Speed*. The categorization depended on the perspective that we took: for example, in the first step we gathered all the challenges concerning interactions under the ‘Interaction’ area of improvement and its sub-categories. In the results we show how ‘interaction’ factors (with negative influence on speed) were outnumbering the other factors, motivating the follow-up survey.

3.4.2. Quantitative analysis. We performed statistical analysis on the answers to the close-ended questions in the questionnaire, using the tool SPSS. The purpose was to validate and rank the challenges, for which we used frequency analysis, and to correlate the answers to roles and companies by using Pearson chi-squared tests. We could not perform more parametric tests because the size of the sample was not large enough to provide meaningful statistical evidences. The results are shown in Table III.

3.4.2.1. Frequency analysis. We have used frequency analysis for the *recognition* of a factor. We have grouped the factors in *strongly* recognized and *weakly* recognized, depending on the ratio of the participants answering ‘yes’ or ‘no’. However, when the number of yes and no was similar, we considered the factors as *controversial*, which means that even if the factors are strongly or weakly recognized, there was contrast in the answers. Combining these two categorizations, we had four categories: *strongly recognized*, *strongly but controversial*, *weakly but controversial*, and *weakly recognized* factors.

We have also explored the *perceived spread* of a factor based on the respondents’ experience of the issue being not only local (for example, present in the respondent’s team) but affecting other parts of the organization close to the respondents. In order to evaluate the spread of the issues through the organization, we had to weight the four answers provided by the respondents. We obtained ordinal data similar to a Likert scale without a middle-point. We have weighted the answers with the following weights: zero was mapped to no, one to ‘yes in other units/teams’, three to ‘yes in my unit’ and four to ‘yes in my unit and in other units’ (1 + 3). We have weighted the answer *yes in my unit* as three times *yes in other units*: this way we ‘trusted’ the direct experience more than the participants’ perception about the presence of the factor in other parts of the same organization. We have emphasized the polarity between the no (zero) and the yes (3): in this way the means around three would show strong recognition, means around zero strong negation, and means around two would show high polarity in answers (not many neutral answers). The one point assigned to *in other units* weakly influences the means and is useful to give us an idea of how much the factor is perceived as spread throughout the organization. We could have assigned a higher weight to *in*

Table III. List of challenges ranked by recognition, spread, and their dependence on context or on role.

CHALLENGE (abbr.)	ID	RECOGNITION		RANK WITH SPREAD		DEPENDS ON CONTEXT	DEPENDS ON ROLE
		%		MEAN			
Developers and system engineers are not co-located Processes/ways of working not suited for the kind of product "Invisible" satellite unit No time to improve parts of software shared among projects. Lack of documentation/communication, long warming-up Not sufficient communication for reuse Different attitudes and values of distributed teams Erroneous assumptions on reusable software Project-related defects	Q04	93.9	Strong	3.52			
	Q01	83.3		3.13			
	Q08	84.2		3.11			
	Q07	85.3		3.00		x	
	Q14	86.1		2.94			
	Q09	84.8		2.94			
	Q12	80.6		2.72			!
	Q05	77.8		2.67			x
	Q03	75.0		2.44		x	
	Q15	64.7		2.38			
Abundant documentation does not help understanding the code Reuse individual initiative System architecture artifacts not clear enough Disagreement on features in reused software Loss of knowledge about a reused framework's variation points Different favorite tools in the same team Estimation of new features without implementations constraints Processes/ways of working not suited for the project Leaders' mindset is not open to listen Developers are too constrained by system engineers on design An internal interface had to be exposed to other units	Q11	61.1	Strong, Controversial	2.14			
	Q19	55.6		2.08		x	
	Q22	60.0		2.03			
	Q23	54.5		1.91		x	
	Q18	54.3		1.89			
	Q06	52.8		1.86			!
	Q02	54.3		1.86			
	Q16	50.0		1.64			
	Q20	52.8		1.64			
	Q21	50.0		1.53			
Different favorite programming languages in the same team Team's lack of will to integrate a common component Development unit was forced to integrate a common component	Q17	36.7	Weak, Contr.	1.33			
	Q13	38.2		1.26			!
	Q10	29.0		.84			x
		⊗	0.0	0.00			

The challenges are marked with an x. The mark '!' signals the presence of evidences in the data that need further validation

Table IV. Number of answers for Q03 by company, followed by the chi-squared result.

Q03- Project-Related Defects				
Answer Company	No	Yes, my other units	Yes, my unit	Yes my and other units
C	6	1	3	1
B	2	1	7	11
Pearson chi-square				0.021

Table V. Number of answers for Q07 by company, followed by the chi-squared result.

Q07- There is no time to improve parts of software shared among projects				
Answer Company	No	Yes, my other units	Yes, my unit	Yes my and other units
C	4	1	1	4
B	0	0	8	13
Pearson chi-square				0.005

Table VI. Number of answers for Q23 by company, followed by the Chi-Square result.

Q23- Loss of knowledge about a reused framework's variation points				
Answer Company	No	Yes, my other units	Yes, my unit	Yes my and other units
C	4	0	2	0
B	5	1	1	7
Pearson chi-square				0.1

Table VII. Number of answers for Q05 by role, followed by the chi-squared result.

Q05- Erroneous assumptions on reusable software				
Answer Role	No	Yes, my other units	Yes, my unit	Yes my and other units
System eng.	1	0	1	5
Designer	4	0	5	4
Manager	0	2	3	5
Tester	3	0	1	1
Pearson chi-square				0.1

Table VIII. Number of answers for Q10 by role, followed by the Chi-Square result.

Q10- There is no time to improve parts of software shared among projects				
Answer Role	No	Yes, my other units	Yes, my unit	Yes my and other units
System eng.	4	0	2	0
Designer	11	0	1	0
Manager	2	2	1	2
Tester	5	0	0	0
Pearson chi-square				0.027

other units, and the results would have given a stronger result on the spread of the factors, but we have chosen to be cautious.

3.4.2.2. Chi-squared tests. To discover if the answers on the factors depended on the company, we have used cross tabulation, and performed a Pearson chi-squared test to check if the distribution of the answers changed with respect to the companies B and C (we could not use A because of the few respondents). For factors that gave a significant response ($p\text{-value} < 0.1$), we could find evidence that there were differences in the expected distribution. We performed the same test using the roles as categories. Finally, we combined the categories, roles, and companies, in another cross tabulation, but the single cells contained too few entries to perform a useful analysis. The selected and detailed data relevant for the explanation of the results are displayed in Tables IV–VIII.

4. RESULTS AND ANALYSIS

4.1. Interaction challenges hinder the achievement of business goals based on speed in all studied contexts

With the first step we found the challenges hindering the achievement of business goals based on speed. The qualitative investigation brought to light 114 factors [8]. After the categorization of the factors, we obtained a distribution of the factors in the areas of improvement showed in the bar charts in Figure 3. The diagrams display which areas the informants were most concerned with, within a single company and cross company.

The results show that interaction challenges had an impact on first development speed, replication speed, and evolution speed:

- **First Deployment Speed:** when a set of features is released for the first time, the speed is affected by the interaction speed among the teams that have to integrate the features. This kind of speed has an impact on *hitting the market fast* to anticipate the competitors. Fast deployment speed also *shortens the loop in market testing*.
- **Replication Speed:** when a feature is embedded in a previous release, interactions are needed between the team responsible for the new features and the teams that had developed the former ones. Replication increases ROI when the effort made for the first deployment speed is spread on the *release of new products and services based on the existing software*.
- **Evolution Speed:** when a feature needs to evolve after its release, the changes will affect other features, which requires interactions. The speed in *reacting upon a change request* can be critical for gaining the trust of the customers.

The even distribution of the 30 factors in the *interaction* column of Figure 3 (12, 10, and 8 for cases A, B, and C), emphasizes that interactions is a common concern at all sites.

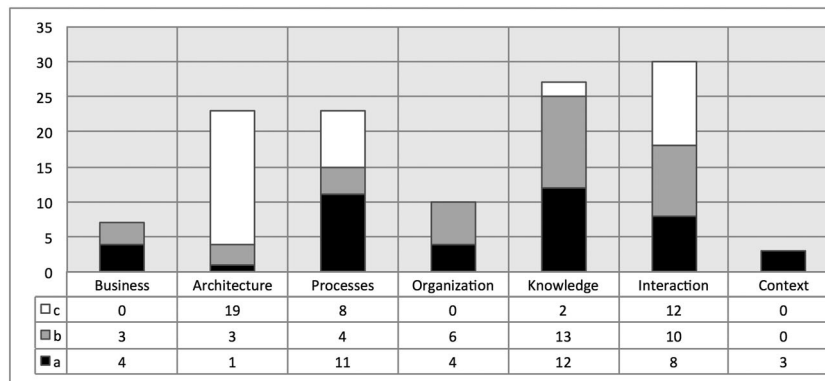


Figure 3. Bar chart of all challenges. Interaction is the most populated category and also contains an even distribution of the challenges.

These results called for further investigation of intra-group interaction challenges. The category containing such factors was the most populated one, which showed its importance for the respondents. This was also the category with the most evenly distributed factors among the studied sites, which suggested a high degree of generalizability of such factors.

4.2. Validation and prioritization of interaction challenges

In the second step we conducted a survey to validate and prioritize the discovered interaction factors.

Through a quantitative analysis of the answers, we produced an ordered table with the factors, their recognition in terms of frequencies and their perceived spread in the companies. The analysis of the data also highlighted factors recognized by the respondents in some contexts but not in others, and factors recognized by some roles but not by others. The results are summarized in Table III.

The second column from the left (*Recognition*) displays the percentage of respondents (calculated on the valid answers provided) that have recognized that factor. All the factors are recognized at least by 29% of the participants. In the middle column we highlighted the level of recognition (*strongly*, *strongly but controversial*, *weakly but controversial*, and *weakly recognized*) with bold borders grouping the rows of the factors included in each cluster. Nine factors, with more than 75%, are strongly recognized. The factors with more than 50%, another 11, are strongly recognized but controversial, which means that there are more yes than no, but we found some disagreement in the answers. The last three factors are weakly recognized but controversial, which means that the number of no is larger than the number of yes, but some of the respondents considered the challenge present in their context. None of the factors were considered weakly recognized.

In the middle column (*Rank with Spread*) we show the spread calculated using the means of the answers weighted from zero to four: such results, together with the recognition, illustrate how much the respondents perceive the spread of the factor in other parts of the organizations close to them. This partially changes the order of the factors, as we can see for Q14 and Q01: the former one has a higher direct recognition rate, while the latter one is acknowledged by less respondents than Q14 but is been considered more widespread within the company. Notice that the spread does not influence the previous categorization of the factors (*strongly*, *weakly*, etc.).

In the last two columns from the left (*Depends on Context* and *Depends on Role*) we have marked the factors with an 'x' when we have statistical evidences that the factor has the property.

4.2.1. Context-related challenges. The distribution of the answers with respect to the respondents' company did not change for 20 of the factors. This means that we could reject the hypothesis (as explained in the Section 3.4.2.2, the significance was 0.1) that the recognition of most of the factors depended on the company of the respondents only for three of the factors (Q03, Q07, and Q23). The results are shown, respectively, in Tables IV–VI. These challenges are reported in Table III with x on the column *Depends on Context*. In the followings we describe the differences among the companies.

Q03: Project-related bugs or defects

For factor Q03 there is a great difference between company B and C: the respondents in company C do not recognize the factor, while the ones in company B strongly recognize it.

Q07: There is no time to improve parts of software shared among projects

For factor Q07 the respondents in company B strongly recognize it, while the other ones stand in the middle. This shows that this factor is strongly present in company B while is controversial in the other companies. With respect to Table III, this partially weakens the strong recognition of Q07 because the mean is influenced by company B.

Q23: Loss of knowledge about a reused framework's variation points, for example a framework created some years before.

In company B more than half of the informants (eight) strongly recognize this factor. Even though the result is 'strong but controversial', it differs from company C, where only two respondents recognized the factors.

4.2.2. *Different roles' view on some challenges.* We found that the answers changed for only two factors (Q05 and Q10) when given by different respondents with different roles. The results are shown, respectively, in Tables VII and VIII.

Q05: *There is an upcoming product. Erroneous assumptions have been made on what part of the existing software can be reused and/or adjusted, causing inaccurate budget or resources allocation (for example time or workload)*

For this factor, managers and system engineers expressed a clear propensity to recognize the issue, while designers and testers gave more controversial answers.

Q10: *A development unit was forced to integrate a common component (shared with other units). This caused communication problems, and now the unit is not willing to integrate new common components.*

4.3. Boundary spanning

In the previous sections we have identified and ranked the interaction challenges. In Figure 4 we show the critical boundaries in need of interaction improvement. As explained in the methodology section, we list the spanning activities related to the mitigation of interaction challenges at each boundary. Each activity is defined according to a specific boundary, a frequency (*project*, *iteration*, and *ad-hoc*) and involves a coordinator and a spanning object.

In the following section we perform an in-depth analysis of the critical boundaries and the spanning activities that are currently needed for avoiding the interaction challenges.

4.3.1. *Prioritization of boundaries for spanning activities.* The rank of challenges combined with the qualitative analysis of the textual answers show a number of critical boundaries in need of spanning activities, between the agile team and other social groups:

Team - System engineers: Q04 is the most recognized factor, almost by all the respondents (93%, which include many system engineers): system requirements and architecture should be continuously discussed between system engineers and the agile team. This is especially important for

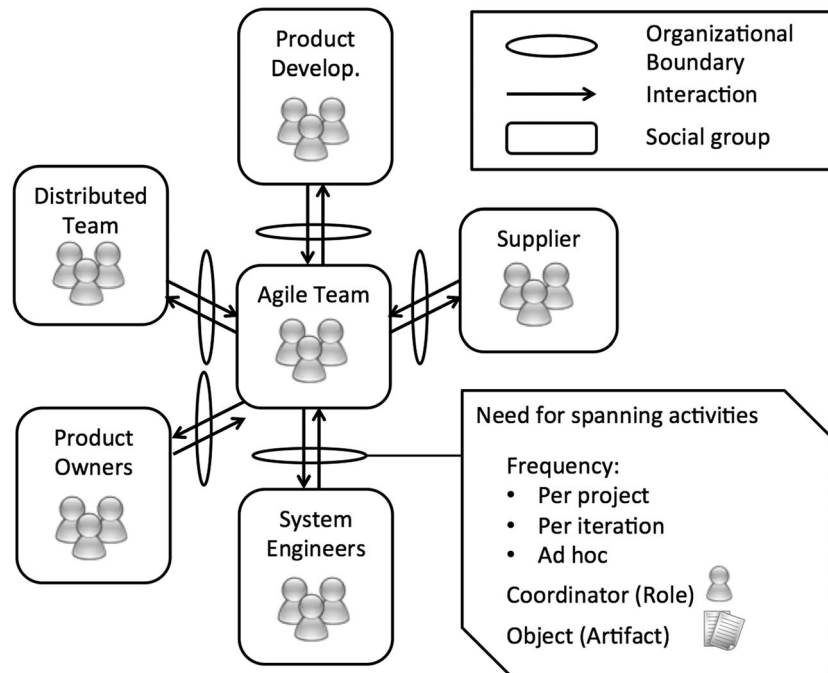


Figure 4. Critical organizational boundaries in need of spanning activities. Each spanning activity has a frequency, a coordinator and it might involve an object.

software reuse, which has to be recognized at a system level and mapped to the agile organization in an optimal way.

Team - Product development and management: The respondents' answers about factor Q01 and Q02 (process-related communication) are quite different. It seems that ASD suffers particularly when the agile team needs to interact with the product development rather than with the project. This aspect might be connected with the kind of developed product (i.e., embedded systems): hardware's design and development process follow a more waterfall-oriented model, which conflicts with the iterative one claimed by ASD. In general, software development is bound to the overall product development process, which has longer iterations than the ones prescribed by ASD and therefore hinders the responsiveness that would be gained by employing ASD at a team level. However, we cannot say if this is a real constraint or a factor related to the legacy of the product development in place. In the latter case, we hypothesize (supported by the qualitative answers) that a change in the product development process to make it more ASD-friendly (for example, by providing frequent interfaces for strategic input) would foster the interaction speed of teams with the product management.

Team - Distributed teams (Q08): The speed of the agile team might be hindered by dependencies to other distributed teams. This includes suppliers commissioned to develop outsourced software components but also teams distributed in several buildings. Especially, when the quantity and the channels of communication are constrained by contract terms such as time intervals or approval requirement, speed is hindered by the occurrence of strong delays. The same, but less drastic issues, occur by mismatches in processes, practices, attitudes and values. Frequent meetings on-site and social network activities would ease this interface.

Team - Other projects' teams: Q07, Q03, and Q09 are strongly recognized, but the results have been stronger for a specific context (Q07, Q03 for case B). The isolation of the projects, especially in terms of budget and resources, create silos that hinder reuse across the projects. The exclusive focus on one project by the team leads also to hindering the communication (and speed) among teams in parallel projects: this creates inefficiencies, especially if there are dependencies among the projects, and strategic reuse always brings such dependencies. Solutions such as relocation in other projects have been proposed in the qualitative answers, to spread knowledge of the system and to gain acquaintance among the employees.

Team - Product owners: Factor Q05 shows that software reuse can be erroneously considered during the selling process (or the marketing scope). In the qualitative answers, respondents specified that no resource allocation was estimated when reusing *similar* components. However, the business value of identical components seems to be much higher than that of the similar ones. This suggests that full reusability of components should be checked by the Product Owners. This would encompass communication with the involved teams responsible for the components. The strong recognition of this factor mainly by management and systemization shows that designers and testers are not fully aware of strategic and business related goals. This suggests that if the agile team is not guided continuously by strategic inputs, it might incur in sub-optimal decisions.

4.3.2. Boundary spanning for each project. Once we identified the challenges and the critical boundaries, we analyzed the qualitative answers of the participants in order to find spanning activities that were used or missing for the whole project.

Several informants mentioned the need for activities at the beginning of the projects.

'Everyone start off being very busy with their own stuff and do not have time to talk to others. Eventually this leads to a crisis in the project where the schedule slips and integrations fails.'

The most common means to achieve synchronization is the organization of workshops. The data suggest that the workshops should be focused especially on the following actors and activities.

Architecture between the team and the system engineers (architects): architecture is a synchronization mechanism and is important for achieving platform and component reuse among several projects in all investigated cases. The architecture is also seen as a *boundary object*, and the production of such object needs to be agreed upon. The current implementation of ASD seems to suffer from the lack of suitable spanning objects, for example, the architecture documentation. This documentation is currently not fulfilling the needs of the stakeholders.

'Better coordination and planning of how architectural changes should be introduced in a product with high feature growth. The changes should be introduced in steps and following a plan agreed to by all concerned parts'

A workshop in the beginning of the project would help agreement on the stakeholders' 'requirements' for the necessary boundary object (both the team and the architects).

'Some defined level of documentation describing the system/subsystem for a newbie, requirement engineer to more easily get into/understand the system', but 'have less complimentary documentation, in best case only some pictures to explain the overall idea'.

Furthermore, the discussion would increase the team's awareness about the product and product line (through the architects' view) and ease the follow-up communication with the architects with the increase of personal acquaintance. No coordinator role has been identified, but we hypothesize that the architects would be the coordinators for this activity.

Expectation between distributed development teams and between the team and the suppliers: according to the informants, a clear challenge is, for a team, to know what to expect from other teams. Teams need also to agree on future spanning activities in order to avoid hindering each other during development and to find suitable points for synchronization in the customized processes.

Negotiation and awareness between teams, architects and product owners: what the respondents lament is the decision making based on wrong assumptions in the scope analysis which would be reflected on the pressure suffered by the team to meet deadlines. Activity tools such as planning poker is used *within* the team, for example for estimating the cost of feature development, but we found the need for a workshop covering the feasibility and the expectations (negotiation) on higher level development capability, such as architecture goals (including other projects) and time-related penalties to be agreed with the customers. This kind of workshop would also increase information about business strategies in the development team, making them closer to the customer perspective.

4.3.3. Boundary spanning for each iteration. Spanning activities aligned with iteration frequency can be used as a synchronization mechanism among several teams and between the team and other kinds of organizational groups [13]. However, often the respondents mention the lack of coordination among interacting teams and between the team and other groups.

Process mismatch among teams: A major challenge in the former case is the lack of organized opportunities for the teams to meet among themselves. Such problem is also connected by the respondents to the mismatch of processes: the processes are usually customized by the teams; for example, the development of two features might start in different times and the iterations and development phases within the iteration might not be synchronized. Such challenge has been especially emphasized when an agile team had to be in contact with external suppliers not complying with ASD (cases A and C).

Process mismatch between the team and other groups: As for the coordination between the team and other groups, a main challenge is the mismatch of processes between the agile team, organized in iterations, and the rest of the organizational groups/other organizations, which is usually not. The product development for all the cases is bound to the creation of the actual physical system on which the software has to be deployed. Consequently, the agile team lacks interaction with other engineering experts (e.g., system engineers), product management, and other resources that follow a different process and therefore are not available. As an example, it might happen that a feature is not tested as soon as it is been developed.

'A feature should be verified as soon as it has been implemented (and not a few months later) to get fast feedback, save time and prevent a lot of bugs at the end of the project. - When creating parallel implementation tracks in the same product/project there should be a plan for how the tracks should be merged together'.

Spanning objects: generic process guidelines to be reused.

Coordination role: The spanning activities would include the presence of a coordinator in the other groups that would facilitate the communication with the agile team by being available at the end of each sprint with the necessary information. For example, a system engineer or an electrical engineer should be appointed for the interaction with the agile team.

Coordination among the teams: Many respondents argue that there is a need for making the team responsible for their interaction with other teams. Such interaction is usually because of socio-technical dependencies. In some cases such interaction might be regulated through social links developed among the teams, through communities of practices [30] or might be initiated by a Scrum Master (e.g. using Scrum of Scrums [10]) by organizing inter-team meetings each iteration. However, the informants usually consider part of the documentation necessary for coordination, because only face-to-face interactions seem not to be enough in a large environment. *'Insert documentation stories into the backlog. Include documentation into definition of done'.*

Spanning objects: The activity is to include the creation of documentation as a task/story in the backlog. Such boundary activity would assure the prioritization of the boundary object (documentation) creation through the use of another boundary object, the backlog itself.

4.3.4. *Ad hoc boundary spanning.*

Shared responsibility of integration: The respondents highlighted the need for having clear responsibilities for the integrated parts of the software. Currently there is a lack of clear reference about who should be responsible for implementing bug fixes, for improving the code (including architecture) and for explaining past decisions. The integrated 'whole' might be a component (in case features spread to several components) or a set of connected functionalities.

'Take responsibility for "your product", '[...] very clear who is doing what and when everybody is supposed to deliver what and to whom. The overall objective of each delivery and the limitations (not implemented functionality) should also be communicated to all involved parties'.

'Promote the attitude of taking responsibility for the WHOLE chain of functionality'.

Respondents suggest a spanning activity consisting of monitoring and recognizing such situations in which responsibility could 'fall between the cracks', and reacting by defining responsibilities through a meeting with the involved actors.

'Review interfaces between different processes e.g. System design - SW Design to enable a more continuous flow of information'

However, also in this case, a coordinator has not been identified by the respondents. Such an activity could probably be carried out at each iteration, but according to the respondents, it is enough to monitor and react in an ad hoc way.

5. DISCUSSION

Our research focused on 23 challenges related to achieving business goals dependent on speed. The emphasis is on interactions across organizational boundaries, specifically between the agile team and other parts of the organization. Our findings are focused on large organizations developing embedded software and employing ASD.

The results show several and widespread interaction challenges influencing business goals dependent on speed. Highlighting the most recognized and spread interaction challenges would help practitioners in selecting the targets for investing resources and would provide guidelines for further research. Showing the relationship between challenges and context warns practitioners about known issues in applying ASD in large companies developing embedded software. Showing the different views among the roles on some factors reveal possible existing conflicts or lack of awareness about some of the challenges, for example, the lack, in the teams, of strategic goals that need to be communicated by product managers and architects. Highlighting which practices are implemented or missing would help organizations in defining a coordination strategy that would include spanning

activities, objects and coordinators in order to mitigate interaction challenges and therefore eliminating delays in the reaching the business goals.

In the following sections we discuss the generalizability and contextualization of challenges and activities. Then, we compare the results to the cases found in the literature [13] in order to highlight our contribution with respect to related work and to answer RQ4. Finally, we discuss the main limitations and threats to validity and present more related work.

5.0.1. Generalization and contextualization of challenges. The results show an overall recognition and spread of the challenges. Twenty challenges have been strongly recognized, of which 11 were still controversial. Only three challenges were weakly recognized but controversial. No challenges were recognized as weak without any controversy. These results allow us to make a further step towards the generalization of the found challenges. In fact, there is convincing evidence that companies B and C equally agreed on the recognition of the challenges. With less evidence, we can consider company A on the same line. This means that Table III is equally informative for the companies.

As for Q03, Q07, and Q23, the recognition varies according to the context, and Table III shows this together with the ranking of the challenges. These factors are strongly related to context B. In this company projects were isolated, hindering the employment of an overall strategy for reuse. The distinctive parameter related to context B is related to the business model: individual customers with different needs and different time constraints may strongly hinder the investment needed for the creation of a software product line. This is shown also by the recognition of Q23 mainly by case B: according to the qualitative data, the business value of the ad-hoc reuse was overestimated. If the component was not reused exactly as originally implemented, even slight changes would bring a lot of work to re-understand the implementation and to test the small changes that could affect the behavior.

5.0.2. Generalization and contextualization of activities. The consistency among the respondents' answers for these three cases is quite high. We have not found controversial statements among the respondents, and the codes and categories used for this analysis are linked to citation by at least two respondents. The only misalignment in the statements was related to the balance between the need of increasing boundary-spanning activities and team isolation. The actual amount of time spent in interaction is not known and the spanning activities need to be limited in order to allow the team to focus. Such topic, in our opinion, requires further research. The beginning of the project needs more time dedicated to spanning activities (e.g. communication of strategic goals to the team) and the creation of coordination objects (e.g. documentation shared by the teams). Even though it seems to be against the avoidance of a big upfront investment, as suggested by the ASD principles, other experiences reported in literature [31] refer to a 'Sprint Zero', in which the team needs to understand strategic goals. The iterations are good opportunities for coordination and for instantiating or carrying out spanning activities.

Our results are not limited to one case only, but evidences from all three cases support most of the spanning activities reported in the results section. Such activities would help the creation, both for researchers and practitioners, of frameworks dedicated to embedded software development for complementing existing ASD practices. Especially, the following areas are in need of improvements: the starting phase of the project, the development and maintainability of software architecture artifacts, the exchange of information between the team and the product owners, and the interaction with distributed teams.

A relevant practice especially related to company B would be the creation of inter-project spanning activity for helping the agile teams, involved in several projects, in communicating similar strategic goals concerning software reuse. This activity and similar activities remain a challenge to be further studied, because the respondents could not propose a suitable solution. What we can report from our results is that if the business situation of the company includes non-synchronized customers, this kind of spanning activity should be an ad hoc activity. Coordinator and roles need also to be further investigated.

A major challenge in the current organizations is to find suitable roles: groups differing from the agile team appear to have different views and mindsets, which do not necessarily comply with ASD. This is hindering the development of boundary spanning activities and objects.

5.0.3. Comparison between embedded software development and pure software development. In order to show the contribution with respect to RQ4, we highlight the differences between our domain (large product companies developing embedded systems) and the one described in Strode *et al.* [13]. The paper is focused on coordination strategies in ASD, which partially covers spanning activities as well. This paper is also the only one found by the literature review related to spanning activities. However, the companies analyzed in Strode *et al.* [13] are not large companies, and they do not develop embedded software. We show which areas of improvement with respect to which contextual factors are different in our studied domain, and we claim that we need further research dedicated to study new boundary spanning activities for such combination of context and areas.

We recognize four major contextual factors that are different between our cases and the ones described in literature:

- *Reuse*: The presence of heavy reuse (platforms and components) across the projects versus stand-alone projects.
- *Product development*: An overall product-development process is present in our domain but not mentioned in the cases in Strode *et al.* [13].
- *Customer*: In our domain the same product(s) is developed for multiple customers, who, in most cases, are not available (for example, the product is developed for a generic market). This is different from the cases studied in Strode *et al.* [13], where the organizations developed one product for a specific customer. In particular, in our cases we can differentiate between synchronized (cases A and C) and non-synchronized customers (case B).
- *Other disciplines*: In our domain other disciplines need to be connected to software development, such as electrical engineering and system engineering, which brings in more stakeholders for the team than in Strode *et al.* [13].

The major differences in the results are the needs for more boundary-spanning activities, objects, and coordinators in our cases, especially in the following areas:

- *Software architecture*: Spanning activities, objects, and coordinators for this purpose are strongly needed by the respondents, while they are not mentioned at all in Strode *et al.* [13]. We can relate this difference directly to the reuse factor, because software architecture is the main coordination mechanism for reuse.
- *Processes*: In Strode *et al.* [13] it is mentioned that the iterations might be used as synchronization mechanism. However, when a number of different processes *mismatch* but need to *interact*, there is a need for more spanning activities and coordinators. We can relate this issue with the context factors ‘product development’ and ‘other disciplines’, which we found are the main causes for having processes that mismatch.
- *Shared responsibilities*: In our results there are many references to the need of spanning activities for managing shared responsibilities across teams, while they are only briefly mentioned in Strode *et al.* [13], without in-depth research focus. Such responsibilities are mainly linked to the area of software architecture and therefore to the reuse factor (and therefore the replication speed goal), because the responsibilities are shared for integration and for the common understanding of the system.
- *Expectations*: Spanning activities are mentioned in Strode *et al.* [13] for connecting with the customer that is not on site. In our cases, not only the customers are not on site but there are also many at the same time for a given product (linked to the same contextual factor). This increases the need for boundary spanning activities and roles, in order to assure the convergence in understanding requirements among the teams.

Table IX summarizes the need for new boundary spanning activities in embedded software companies employing ASD, which would avoid interaction challenges and would, therefore, improve speed.

5.0.4. Limitations and threats to validity. This study has limitations: the ordinal data gained by the close-ended questions are based on the employees’ perception about the presence of the issues in

Table IX. Comparison between embedded and pure software development according to current literature.

Contextual Factor Area	Reuse	Product Dev.	Customer	Other disciplines
Architecture	x			
Processes		x		x
Shared responsibilities	x			
Expectations		x	x	

The black cells with 'x' show that we need novel spanning activities in the areas on the left column because of specific contextual factors.

their unit or in other units, which cannot be considered a precise measurement. However, we have used appropriate statistical tools such as cross-tabulation and chi-squared analysis of distribution (Tables IV–VIII). Randomization of the sample cannot be completely assured, because the respondents have been conveniently selected by the companies themselves. However, we have explicitly recommended our contacts in the companies not to influence their choice.

The qualitative information is based on employees' statements and may be biased. However, the statements do not usually conflict with each other. On the contrary, the codes are often supported by quotations from several respondents and from several cases. Moreover, we have mainly discussed and drawn conclusions on factors that were strongly recognized, minimizing the possibility errors. The hypothesis and suggestions for improvements have been selected among the most cited improvements coded during qualitative analysis.

We consider the threats to validity as presented in Runeson and Höst [27] and Wohlin *et al.* [32]. Possible threats to construct validity are as follows: evaluation apprehension, mono-operation bias, and background influence. We have handled these with the anonymity of the respondents, collecting data from three companies, and we made sure that the respondents represented several roles, from managers to programmers. We limited the threats to conclusion validity such as instrumental flaws and influence posed on the subjects by running pre-tests with colleagues from academia and contacts in industries. As for internal validity, each respondent has taken the survey just once, while boredom was avoided by the possibility of interruption and continuation in a later moment. As for external validity (generalizability), the chosen cases allow us to generalize some of the factors and activities with respect to the commonalities in their contexts. By studying three cases we have partially limited external validity.

5.0.5. Other related work. Lindvall *et al.* [33] collected experiences from large companies (similar setting) in order to study the integration of ASD with standard processes already in place. The focus was in tailoring XP to suit the standard process and quality management. Under the category of 'Cross-team communication support' the authors mention the presence of challenges in inter-team communication as an open issue. We have found more challenges and studied them in depth.

Karlström and Runeson studied the application of XP in stage-gate project management [34]. They conclude that such combination is feasible, but the agile team should interface with the gates, while at the same time the project management should adapt to support informal communication and documentation with the XP team. This is in line with our results: agile teams need spanning activities with product management and system responsible. The study [34] adds the suggestion, for the agile team, to adapt to the stage-gate model to coordinate with other teams.

Kettunen and Laanti [35] studied the large-scale organizational agility and proposed a framework for a company to embed an agile software project in a successful way. In the paper, the authors

stress the need for an external process adaptable to the agile team, which confirmed our findings. However, the paper is not focused on other interactions that we have studied.

The paper by Lee [36] is an experience report on transitioning Large-scale project into agile. However, the paper is mainly focused on the intra-team perspective. As for the inter-team interactions, the authors propose an agile method for mail management and discuss the competition occurring among teams as a result of the introduction of metrics for assessing ASD.

On the basis of a literature review on ASD, Turk, France, and Rumpe give their perception about the limitations of ASD [37]. They mention *Limited support for building reusable artifacts*, describing the importance of reuse practices, which bring especially long-term benefits. However, the authors do not investigate the issue in depth and do not provide any guideline. Our work focuses on this problem and analyzing the inter-group interaction factors related to it.

In Pikkarainen *et al.* [9] the focus is similar to ours, but the study has been carried out in one organization with several context, performing qualitative data analysis, while we have surveyed the challenges also in a more quantitative way in order to have a prioritized list of the most important ones. Moreover, we have studied specific interaction challenges related to several kinds of speed, which is not included in the scope of Pikkarainen *et al.* [9].

In Paasivaara and Lassenius [38] the authors present a set of interaction practices for Global Software Development. However, the study is not related to ASD. Some practices may be used to ease the factors studied in our paper, but such practices need to be adapted to an ASD environment.

6. CONCLUSION

In this paper we have studied inter-group interaction challenges influencing business goals dependent on speed. In particular, we have been focusing on large companies developing embedded software and employing ASD. We have thoroughly investigated the problem with a two-year case-study involving three large companies and several employees with different roles. We have employed both qualitative and quantitative research methods, with the aim of supporting the claims with a high degree of evidence triangulation.

We have studied 23 interaction challenges (Figure 3), most of which have been strongly recognized by the practitioners and they have been regarded as widespread in the companies (Table III). Our results help practitioners in selecting targets for investing resources and provide guidelines for further research (RQ1 and RQ2).

We have shown the relationship between challenges and different contexts, which warns practitioners about known issues in applying ASD in specific contexts, such as embedded software development and companies developing product lines (Table IX). The main result is the recognition of the need for implementing more boundary spanning activities for specific areas (RQ4). Especially, in cases like company B, where reuse is an imperative business goal (replication speed) and has to be combined with the presence of multiple, non-synchronized customers, spanning activities need to be employed to mitigate the challenges between projects. Also, we have showed the different views among the roles on some factors, which reveal possible existing conflicts or lack of awareness about some issues.

Although we have explicitly asked the practitioners to share the best practices in use, we could see how many interaction challenges still miss a suitable solution, especially across specific critical boundaries (Figure 4). We build our results on top of existing research in coordination theory, and we provide guidelines by highlighting which organizational boundaries are needed for improving critical development areas (Table IX) and with which frequency (e.g. each iteration and project). The results underline the importance of further research for defining new spanning activities, objects and coordinators to mitigate interaction challenges hindering speed in embedded software development combined with ASD (RQ3 and RQ4).

Our future research is aimed at developing a framework including activities, roles, and objects that would fill the gap recognized in this study. We are currently investigating the development of such framework in collaboration with several large companies.–

ACKNOWLEDGEMENTS

We thank all the industrial partners in the Software Center.

REFERENCES

1. Dingsøyr T, Nerur S, Balijepally V, Moe NB. A decade of agile methodologies: towards explaining agile software development. *Journal of Systems and Software* 2012; **85**(6):1213–1221.
2. Baskerville R, Pries-Heje J, Madsen S. Post-agility: what follows a decade of agility? *Information and Software Technology* 2011; **53**(5):543–555.
3. Xie M, Shen M, Rong G, Shao D. Empirical studies of embedded software development using agile methods: a systematic review. in Proceedings of the 2nd international workshop on Evidential assessment of software technologies, 2012, pp. 21–26.
4. Martini A, Pareto L, Bosch J. Communication factors for speed and reuse in large-scale agile software development. in Proceedings of the 17th International Software Product Line Conference, New York, NY, USA, 2013, pp. 42–51.
5. Martini A, Pareto L, Bosch J. Improving businesses success by managing interactions among Agile teams in large organizations. Software Business. From Physical Products to Software Services and Solutions Herzwurm G, Margaria T (eds.). Springer: Berlin Heidelberg, 2013; 60–72.
6. Bosch J, Bosch-Sijtsema P. From integration to composition: on the impact of software product lines, global development and ecosystems. *Journal of Systems and Software* 2010; **83**(1):67–76.
7. Dybå T, Dingsøyr T. Empirical studies of agile software development: a systematic review. *Information and Software Technology* 2008; **50**(9–10):833–859.
8. Martini A, Pareto L, Bosch J. Enablers and inhibitors for speed with reuse. in Proceedings of the 16th International Software Product Line Conference - Volume 1, New York, NY, USA, 2012, pp. 116–125.
9. Pikkariainen M, Haikara J, Salo O, Abrahamsson P, Still J. The impact of agile practices on communication in software development. *Empirical Software Engineering* 2008; **13**(3):303–337.
10. Hossain E, Babar MA, Paik H. Using scrum in global software development: a systematic literature review. Fourth IEEE International Conference on Global Software Engineering, 2009. ICGSE 2009. 2009; 175–184.
11. Levina N, Vaast E. The emergence of boundary spanning competence in practice: implications for implementation and use of information systems. *MIS Quarterly* 2005; **29**(2):335–363.
12. Malone TW, Crowston K. The interdisciplinary study of coordination. *ACM Computing Surveys* 1994; **26**(1):87–119.
13. Strode DE, Huff SL, Hope B, Link S. Coordination in co-located agile software development projects. *Journal of Systems and Software* 2012; **85**(6):1222–1238.
14. Blackburn JD, Scudder GD, Van Wassenhove LN. Improving speed and productivity of software development: a global survey of software developers. *IEEE Transactions on Software Engineering* 1996; **22**(12):875–885.
15. Ma Z, Collofello JS, Smith-Daniels DE. Causes and solutions for schedule slippage: a survey of software projects. Performance, Computing, and Communications Conference, 2000. IPCCC '00. Conference Proceeding of the IEEE International. 2000; 373–379.
16. Ma Z, Collofello JS, Smith-Daniels DE. Improving software on-time delivery: an investigation of project delays. 2000 IEEE Aerospace Conference Proceedings, vol. 4. 2000; 421–434.
17. Poppendieck M, Poppendieck T. Implementing Lean Software Development: From Concept to Cash (The Addison-Wesley Signature Series). Addison-Wesley Professional, 2006.
18. Poppendieck M. Lean software development. Companion to the Proceedings of the 29th International Conference on Software Engineering. 2007; 165–166.
19. Wang X, Conboy K, Cawley O. 'Leagile' software development: an experience report analysis of the application of lean approaches in agile software development. *Journal of Systems and Software* 2012; **85**(6):1287–1299.
20. Santos FM, Eisenhardt KM. Organizational boundaries and theories of organization. *Organization Science* 2005; **16**(5):491–508.
21. Eklund U, Bosch J. Applying Agile Development in mass-produced embedded systems. Agile Processes in Software Engineering and Extreme Programming. Springer: Malmö, Sweden, 2012; 31–46.
22. Ronkainen J, Abrahamsson P. Software development under stringent hardware constraints: do agile methods have a chance?. Extreme Programming and Agile Processes in Software Engineering, vol. 2675 Marchesi M, Succi G (eds.). Springer: Berlin/Heidelberg, 2003; 1012–1012.
23. Yin RK. Case Study Research: Design and Methods. SAGE: Thousand Oaks, California, 2009.
24. Dubois A, Gadde L-E. Systematic combining: an abductive approach to case research. *Journal of Business Research* 2002; **55**(7):553–560.
25. Glaser BG, Strauss AL. The Discovery of Grounded Theory: Strategies for Qualitative Research. Transaction Publishers, 2009.
26. Strauss A, Corbin JM. Grounded Theory in Practice. SAGE: Thousand Oaks, California, 1997.
27. Runeson P, Höst M. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering* 2008; **14**(2):131–164.
28. Flick U. An Introduction to Qualitative Research. SAGE: Thousand Oaks, California, 2009.
29. Czaja R, Blair J. Designing Surveys: A Guide to Decisions and Procedures. Pine Forge Press: Thousand Oaks, California, 2005.

30. Kahkonen T. Agile methods for large organizations - building communities of practice. Agile Development Conference, 2004. 2004; 2–10.
31. Lee S, Yong H-S. Distributed agile: project management in a global environment. *Empirical Software Engineering* 2010; **15**(2):204–217.
32. Wohlin C, Runeson P, Host M, Ohlsson C, Regnell B, Wesslén A. Experimentation in Software Engineering: an Introduction. Kluwer Academic Publishers, 2000.
33. Lindvall M, Muthig D, Dagnino A, Wallin C, Stupperich M, Kiefer D, May J, Kahkonen T. Agile software development in large organizations. *Computer* 2004; **37**(12):26–34.
34. Karlstrom D, Runeson P. Combining agile methods with stage-gate project management. *IEEE Software* 2005; **22**(3):43–49.
35. Kettunen P, Laanti M. Combining agile software projects and large-scale organizational agility. *Software Process: Improvement and Practice* 2008; **13**(2):183–193.
36. Lee EC. Forming to performing: transitioning large-scale project into Agile. Agile, 2008. AGILE '08 Conference. 2008; 106–111.
37. Turk D, France R, Rumpe B. Limitations of agile software processes. Third International Conference on eXtreme Programming and Agile Processes in Software Engineering (XP 2002). 2002.
38. Paasivaara M, Lassenius C. Collaboration practices in global inter-organizational software development projects. *Software Process: Improvement and Practice* 2003; **8**(4):183–199.